Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I expect to use several different dataclasses, including my State dataclass. I will use loops in my car designing functions, and the car objects to appear in the canvas will be lists of strings. I hope to improve my ability to loop through 2D lists, as that will be essential to my program working.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

The easiest part will probably be making the lists of strings that represent car models, because all it requires is being able to imagine each character as a colored pixel. Writing functions that pass directly to index and use input as arguments is difficult for me, because it is hard for me to visual how information is passed through the different channels, like TextBox and TextArea, or Buttons.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

Writing the different functions for each customization will be difficult for me, and likely take a lot of my time. I have already started making the car objects (lists of strings) and have tried to begin designing the start page. Making sure all the route functions are passed to correctly will likely take a lot of unit testing, trial, and error.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Getting too stressed out about what the final project will look like distracts me from breaking it down into manageable pieces. The stress makes me sloppy and causes me to avoid working on it all together. I would really like it if we had more in class activities where we have to design a drafter website, focusing on routes and the components that pass to them.

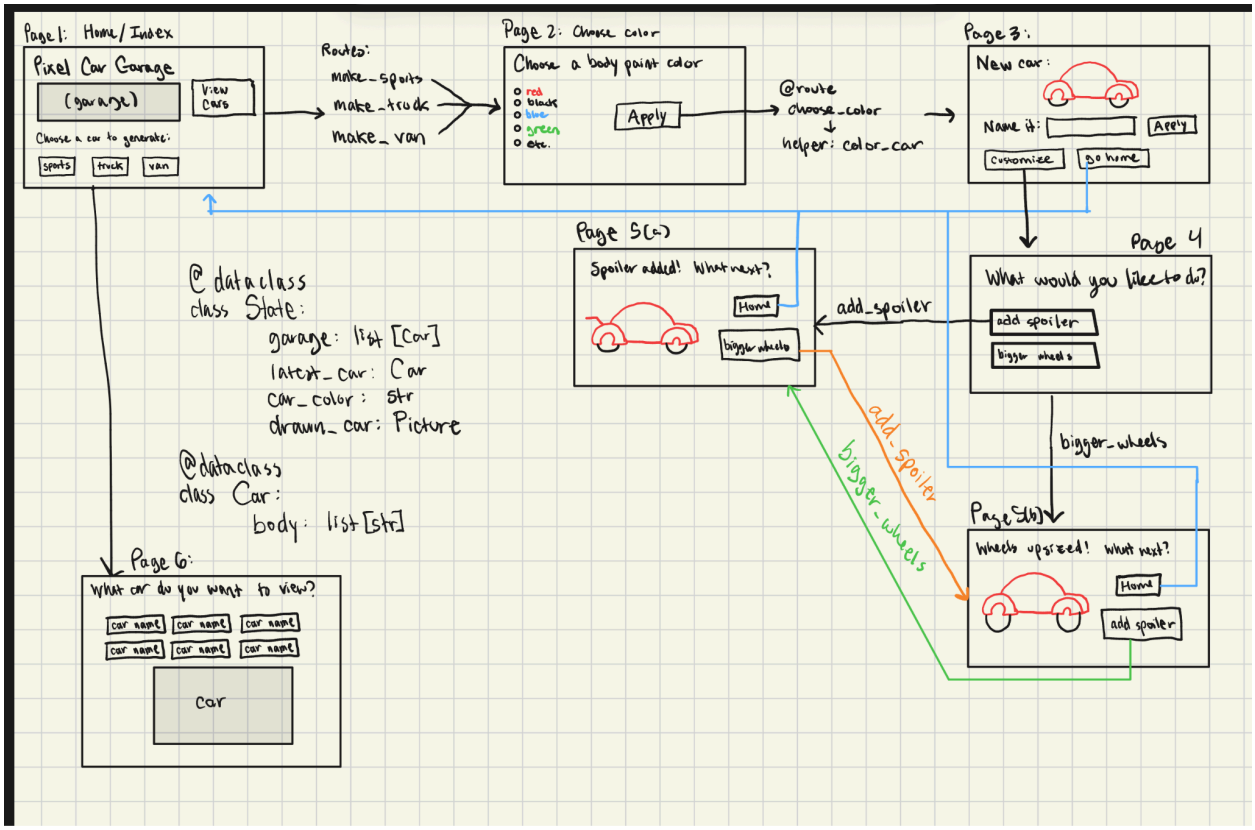
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.


Page 1: Home / Index
Pixel Car Garage
(garage)
View Cars
Choose a car to generate:
sports
truck
van
Routes:
make_sports
make_truck
make_van
Page 2: Choose color
Choose a body paint color
red
black
blue
green
etc.
Apply
@route
choose_color
helper: color_car
Page 3:
New car:
Name it:
Apply
customize
go home
Page 4
What would you like to do?
add spoiler
bigger wheels
Page 5(a)
Spoiler added! What next?
Home
bigger wheels
add_spoiler
bigger_wheels
Page 5(b)
Wheels upsized! What next?
Home
add spoiler
add_spoiler
bigger_wheels
@dataclass
class State:
garage: list[Car]
latest_car: Car
car_color: str
drawn_car: Picture
@dataclass
class Car:
body: list[str]
Page 6:
What car do you want to view?
car name
car name
car name
car name
car name
car name
car

2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	My site is a pixel art generator, specifically for making a virtual car garage. The cars a simple, low-res graphics that can be customized by the user and kept in the “garage” as long as the site is up.
State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	Garage: list[Car] Latest_car: Car Drawn_car: Picture Car_color: str
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	class Car: Body: list[str]
Routes (minimum 7) The names of your route functions	Index Choose_color Color_car bigger_wheels Add_spoiler

	make_sports make_truck Make_van show_car
Pages (minimum 5) A brief description of the pages in your site	Page 1: home/index The first page when you load the app, where the generated cars will be viewable together as you create them. Here is where you decide what kind of car you would like to make next. There is also a button that takes you to Page 6, where you can shuffle through the cars you have made. Page 2: Returned when you select any of the body-style options, and prompts you to select a paint color. The RadioButtonGroup selections are passed directly into the choose_color function, which in turn utilizes the color_car helper. Page 3: The Picture of the newly generated car is displayed to the user, and the option to further customize the car is presented as a button at the bottom of the page. There are also buttons to return home, and a text box to name the car. Page 4: The customization page; here you can choose to add a spoiler or change the wheel size. These options link to functions that change the generated car directly. Page 5a: The newly generated picture of the customized vehicle after selecting the “add spoiler” button, with buttons to either return home or increase wheel size. Page 5b: The newly generated picture of the customized vehicle after selecting the “bigger wheels” button, with buttons to either return home or increase add a spoiler size. Page 6:

	Has buttons for all the cars you have made so far, that you can click to view them.
User Input (minimum 3) Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	Page 1: 1. Body-style Buttons (e.g. “sports”, “truck”, “van”) - purpose is to allow the user to pick what kind of car they would like to generate and add to their garage Page 2: 1. RadioButtonGroup - the paint colors available for users to choose from to color their car Page 3: 1. Buttons: a. Customize: gives user the option to further customize the car with a spoiler or big wheels b. Home: lets user return to the start page 2. TextBox: user can enter a name for the vehicle they just made Page 4: 1. Buttons: a. Add spoiler: lets user modify their car by adding on a spoiler b. Bigger wheels: modified the car with larger wheels c. Home
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	In color_car function: One if and two elif statements function as the main logic behind the loop that translates the car strings into pixels. The conditions of each if statement are dependent on what character is in the string being looped through. The particular sequence of characters is meaningful because it differs depending on which car model the user selects on the home page, and what color they want it to be.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in	View_car will loop through the Car instances in the garage field of state, and identify which button you pressed on Page 6. When it finds the Car that matches the name of the button, it displays that car.

your State attribute. Description of this loop and why it is meaningful to the site	This allows the user to temporarily store cars rather than only being able to see one at a time, which makes the customizations more meaningful.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Me*:* how should i design my index function so the car appears on the page

Chat: For your Drafter project, I'd design your `index` function so that it has two responsibilities:

1. Display the customization options (color, model, wheels, etc.).
2. Display the generated car if one has already been created.

The important thing is that your `index` function shouldn't actually generate the pixel art. That should happen in a separate function. The `index` function should simply display whatever is currently stored in `state`.

1. Think about how the page should behave

When someone first opens your website:

- The customization options appear.
- There is no car displayed yet.
- The user can select their preferences.

After they click Generate:

- Drafter receives their selections.
- Your car-generation function creates the pixel array.
- The resulting array is saved in `state.car`.
- The page displays the generated car.

2. Use a Canvas to display the pixel art

In your earlier implementation, you were using Drafter's `Canvas` component. I'd stick

with that approach because it works well with your character-array design.

Your `index` function could follow this structure:

```
None
@routedef index(state: State) -> Page:    return Page(state, [        Header("Pixel Car Creator"),        # Customization options go here        Button("Generate My Car", "generate"), Canvas("car_canvas", width=300, height=160), Button("Draw My Car", "draw")    ])
```

The important components are:

- `Canvas`: provides the area where your pixel art will appear.
- `Button("Generate My Car", "generate")`: sends the user's selections to your generation route.
- `Button("Draw My Car", "draw")`: triggers the drawing process.

3. Separate generating from displaying

I would keep these as separate responsibilities:

Function	Responsibility
`index`	Displays the website and canvas
`create_car`	Creates the pixel array based on preferences
`generate`	Stores the generated array in `state.car`
`draw`	Reads `state.car` and draws it onto the

canvas

Your `draw` function can use the same nested-loop approach you were using previously:

1. Retrieve the canvas.
2. Clear the canvas.
3. Loop through each row of `state.car`.
4. Loop through each character in the row.
5. Determine the appropriate color.
6. Use `fillRect()` to draw each pixel.

One important detail: Creating a Canvas doesn't automatically draw the car onto it. You'll need to trigger the drawing process after generating the car.

4. One thing I'd like you to think about

Eventually, you'll probably want the car to appear immediately after the user clicks Generate, without requiring a second button.

How could you modify your `generate` route so that it both:

- Saves the newly generated car in `state.car`.
- Initiates the drawing process?

I'd start by getting the separate Generate and Draw buttons working first. Once those work, you can combine the functionality.

This will also make debugging much easier because you'll know whether a problem is happening during car generation or canvas rendering.

Me: what do I need to import to use the brush tool?

Chat: If you're using Drafter's drawing tools, you should import `brush` from `drafter`:

```
None
from drafter import brush
```

If you're using `brush` to draw on a `Canvas`, you may also need the relevant canvas components from Drafter, depending on which drawing functions you're using.

Me: doesn't from drafter import * just account for everything?

Chat: Yes! `from drafter import *` imports all names that Drafter makes available

through its public wildcard imports.

So if `brush` is included in Drafter's public exports, you won't need to import it separately.

However, if you get an error saying `brush` is not defined, it may not be included in those exports.

Me: it says brush is not in module drafter

Chat: Ah, that means `brush` isn't an available name in the `drafter` module. My earlier suggestion to use `from drafter import brush` was incorrect.

Also, `from drafter import *` won't import everything in the entire Drafter package; it only imports the names that Drafter exposes for wildcard imports.

For your pixel car project, you actually have two options:

Option 1: Use Canvas and its browser drawing tools

```
None
import jscanvas =
js.document.getElementById("car_canvas")brush =
canvas.getContext("2d")
```

Here, `brush` is a variable you create yourself. It isn't something you import from Drafter.

Option 2: Use Drafter's `Picture` class

Drafter has a built-in `Picture.new()` and `set_pixel()` method that let you create pixel art directly in Python, without needing Canvas or JavaScript.

Drafter Documentation+1

My recommendation: I'd seriously consider Option 2 for your project. It lets you use your existing nested-list and loop knowledge to draw the car, and then display the resulting picture directly on the page.

You can see the official documentation here: Drafter Picture documentation.

Me: do i have to use picture.new() for every pixel of the car?

Chat: No! You only need to create the `Picture` once. You can then use `set_pixel()` repeatedly to color individual pixels.

The general idea is:

1. Use `Picture.new()` to create a blank picture with your desired width and height.
2. Loop through your nested list representing the car.
3. For each character, determine its corresponding color.
4. Use `set_pixel()` to color that position in the picture.
5. Return the completed picture.

For example, conceptually:

```
None
picture = Picture.new(width, height)for y, row in enumerate(car):    for x, pixel in enumerate(row):        # Determine pixel color        # Set the pixel at (x, y)
```

The important distinction:

- `Picture.new()` creates the blank canvas.
- `set_pixel()` changes individual pixels.

This is actually a great fit for your project because you can reuse the same drawing function for every car model and color!

Me: how do i know what pixel to start at and how to keep drawing on that path?

Chat: The easiest way to approach this is to think of your pixel art as a grid with coordinates, rather than a path that you have to manually follow.

You don't need to keep track of where the previous pixel was drawn. Your nested loops can keep track of the position automatically!

1. Understand the coordinates

Imagine a 5 × 5 grid:

Each square represents one pixel. Coordinates are written as (x, y).

- `x` represents the horizontal position (column).
- `y` represents the vertical position (row).
- The top-left corner is `(0, 0)`.
- Moving right increases `x`.

- Moving down increases `y`.

2. Let your nested loops track the coordinates

Python's `enumerate()` is particularly useful here.

For example:

```
None
for y, row in enumerate(car):    for x, pixel in
enumerate(row):        # Draw pixel at position (x, y)
```

Here's what happens:

- The outer loop moves down through the rows.
- The inner loop moves across the characters in each row.
- Every character has its own `(x, y)` coordinate.

You don't need to manually calculate the next position!

3. How does this work with your car?

Imagine your car is represented by this array:

```
None
car = [    "  RR  ",    " RRRR ",    "RRRRRR",    " OO OO"]
```

Your loops would visit the pixels in this order:

Each character gets visited, including spaces. You can decide to leave spaces transparent.

For example, the first `R` is at `(2, 0)`, and the next `R` is at `(3, 0)`.

4. One question to consider

If your array contains a space (`" "`) representing an empty pixel, do you think you should:

- A. Draw a white pixel there.

- B. Skip drawing that pixel so the background remains transparent.

I'd recommend thinking about this before writing your drawing function because it will affect how your cars appear on the page.

Me: can you give me an example of what a set_pixel() call would look like

Chat: A `set_pixel()` call would look something like this:

```
None
picture.set_pixel(x, y, color)
```

For example:

```
None
picture.set_pixel(2, 0, "red")
```

This colors the pixel at coordinate `(2, 0)` red.

In your car project, you could use it inside your nested loops:

```
None
for y, row in enumerate(car):    for x, pixel in
enumerate(row):        if pixel == "R":
picture.set_pixel(x, y, "red")
```

Here:

- `x` determines the column.
- `y` determines the row.
- `"red"` is the color being applied.

One thing to check: Make sure your version of Drafter's `Picture.set_pixel()` accepts color names as strings. The expected color format may depend on the library version.

Me: what would be an appropriate return value for this test ive written under this code

from bakery import assert_equal

from drafter import Picture

def color_car(car:list[str],car_canvas:Picture,car_color:str):

```
    for y, row in enumerate(car):
        for x, pixel in enumerate(row):
            if pixel == "B":
                car_canvas.set_pixel(x,y,(car_color))
            elif pixel == "W":
                car_canvas.set_pixel(x,y,"gray")
            elif pixel == "T":
                car_canvas.set_pixel(x,y,"dimgray")
            else:
                pass
    return car
assert_equal(color_car(["BBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBWWWWW",
    "BBBBBBBBBBBBBBBBBBWWWWWW",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "BBBBBBBBBBBBBBBBBBBBBBBBBBBB",
    "  TTTTT              TTTTT"],Picture.new(200,200),"black"),
```

Chat: Your current function returns `car`, so your test should compare the result against the original car list.

However, there's something worth considering: What do you actually want `color_car()` to return?

You have two reasonable options:

1. Return `car` — Your function modifies the `car_canvas` but returns the original pixel array. Your test would compare against the original list.
2. Return `car_canvas` — Your function modifies the picture and returns the completed `Picture`. This is probably more useful because you want to display the finished car on your Drafter page.

I'd lean toward option 2 for your project.

One question: Do you intend to use the returned value directly in your Drafter `index` function to display the car, or will you store the `Picture` in `state` first? That will help determine which return value makes the most sense.

Me: i am ignoring state and index right now, just separately coding the function to

worry about later. i fexed the return to be car_canvas

Chat: Perfect! In that case, your test should compare the returned `Picture` against an expected `Picture`.

You have two options:

1. Create an expected picture separately and compare the two pictures, if Drafter's `Picture` supports equality comparison.
2. Test individual pixels in the returned picture, if the class provides a way to retrieve pixel colors.

I'd recommend the second approach because it lets you verify that:

- Body pixels are the selected color (`black`).
- Window pixels are gray.
- Tire pixels are dim gray.
- Empty spaces remain unchanged.

One question: Does your version of Drafter's `Picture` have a `get_pixel()` method? If so, that could make testing your function much easier.

Me: i dont know how to use it

Chat: No problem! Let's figure it out without making things unnecessarily complicated.

First, I want you to try something small:

```
None
picture = Picture.new(200, 200)print(dir(picture))
```

`dir()` displays the available attributes and methods belonging to an object.

Look through the output for something like:

- `get_pixel`
- `get_color`
- `get_pixel_color`

If you find one, tell me what it is.

Alternatively, you can check the official Drafter documentation.

I'd rather help you use a method that actually exists in your version of Drafter than guess at its functionality.

Me: get_pixel

Chat:Great! That means you can use `get_pixel()` to check whether your function colored the pixels correctly.

The general syntax should be:

```
None
car_canvas.get_pixel(x, y)
```

For example, after running your `color_car()` function, you could check:

```
None
result = color_car(car, Picture.new(200, 200),
"black")print(result.get_pixel(0, 0))
```

Since `(0, 0)` corresponds to the first character of your first row, you should get the color you assigned to `"B"`.

Here's what I'd like you to try next:

Run that print statement and tell me what it outputs. Then we can build your `assert_equal()` tests using the actual output format rather than guessing.

Me: it gave 255, 255, 255

Chat: Interesting! `(255, 255, 255)` represents white in RGB color format.

That suggests the pixel at `(0, 0)` is still white, rather than black as you expected.

Let's investigate one thing at a time.

Your first character is `"B"`, so this condition should execute:

```
None
if pixel == "B":    car_canvas.set_pixel(x, y, (car_color))
```

I want you to try printing the pixel immediately after `set_pixel()`:

```
None
car_canvas.set_pixel(x, y,
```

```
car_color)print(car_canvas.get_pixel(x, y))
```

What does it print?

This will help us determine whether:

1. `set_pixel()` isn't changing the pixel.
2. The color argument isn't being interpreted as expected.
3. Something else is happening in your function.

Me: (0, 0, 0)

(0, 0, 0)

(0, 0, 0)

(0, 0, 0)

(0, 0, 0)

(0, 0, 0)

. . .

Chat: Excellent! That confirms your `set_pixel()` call is working.

`(0, 0, 0)` is the RGB representation of black:

- Red: 0
- Green: 0
- Blue: 0

Your function is successfully coloring the body pixels black.

The reason you initially got `(255, 255, 255)` is likely that you checked the pixel before running the updated drawing code, or checked a different picture.

Now let's build your test.

Since `get_pixel()` returns an RGB tuple, you can use `assert_equal()` to check individual pixels.

For example, try testing these three locations:

Coordinate	Expected color

`(0, 0)`	Black `(0, 0, 0)`
`(19, 1)`	Gray
`(2, 6)`	Dim gray

One thing to watch: your canvas is 200 × 200, but your car array is much smaller. That's perfectly fine!

I'd suggest writing three separate assertions rather than comparing the entire Picture object. That way, if something goes wrong, you'll know which part of your car isn't being colored correctly.

Me: (19,1) gave me 128,128,128

Chat: Perfect! `(128, 128, 128)` is the RGB representation of gray.

That means your `"W"` condition is working correctly!

Your test results so far:

Coordinate	Expected	Actual	Result
`(0, 0)`	Black `(0,0,0)`	`(0,0,0)`	Pass
`(19, 1)`	Gray `(128,128,128)`	`(128,128,128)`	Pass

Now try `(2, 6)`, which should correspond to a `"T"` in your last row.

What RGB values do you get?

Me: 105,105,105

Chat: Exactly! `(105, 105, 105)` is the RGB representation of dimgray.

All three of your color conditions are working correctly!

You can now write your assertions:

None

```
assert_equal(result.get_pixel(0, 0), (0, 0,
0))assert_equal(result.get_pixel(19, 1), (128, 128,
128))assert_equal(result.get_pixel(2, 6), (105, 105, 105))
```

I'd also recommend eventually testing an empty space to make sure your function leaves the background untouched.

Me: Im trying to run my code but it keeps saying state is undefined; here is the error message, and also my code:

from drafter import *

from drafter import Picture

from dataclasses import dataclass

@dataclass

class State:

    garage: list[list[str]]

    latest_car: Car

    drawn_car: Picture

    car_color: str

@dataclass

class Car:

    body: list[str]

MATERIALS = {

    "B": "body",

    "T": "tire",

    "W": "window"

    }

COLORS = {

    "Blue": "b",

```
   "Red": "r",
   "Green": "g",
   "Orange": "o",
   "Pink": "p",
   "Black": "x"
   }

CARS = {
"sports": [
   "     BBBBBWWWW     ",
   "    BBBBBBWWWWW",
   "BBBBBBBBBBBBBBBBBBBBB",
   "BBBBBBBBBBBBBBBBBBBBB",
   "  TTTT           TTTT"
   ],
"truck": [
   "         BBBBBBBB",
   "         BBBBBWWWW",
   "         BBBBBWWWWW",
   "BBBBBBBBBBBBBBBBBBBbBBB",
   "BBBBBBBBBBBBBBBBBBBBB",
   "BBBBBBBBBBBBBBBBBBBBB",
   "  TTTT           TTTT"
   ],
"van": [
   "BBBBBBBBBBBBBBBBBBBBB",
   "BBBBBBBBBBBBBBBBBBWWWWW",
   "BBBBBBBBBBBBBBBBBBWWWWWW",
   "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
   "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
   "BBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
   "  TTTTT             TTTTT"
   ]
```

```
}

pix_ht = 10
pix_wd = 10

@route
def index(state: State)->Page:
    return Page(
        state, [
        Header("Pixel Car Garage"),
        "Choose a car to generate",
        Button("Sports Car", "make_sports")
        ])
@route
def make_sports(state: State):
    state.latest_car = Car(CARS["sports"])
    state.garage.append(state.latest_car)
    return Page(
        state, [
        "Choose a body paint color",
        RadioButtonGroup("car_color", ["Green","Blue","Red","Pink","Black","Orange"]),
        Button("Apply","choose_color")
        ])
@route
def choose_color(state: State, car_color:str):
    state.car_color = car_color
    picture = Picture.new(40,40)
    state.drawn_car = color_car(state.latest_car,picture,car_color)
    return Page(
        state, [
        "New Car",
        state.drawn_car
        ])
```

```
def color_car(car:Car,car_canvas:Picture,car_color:str):
    for y, row in enumerate(state.latest_car.body):
        for x, pixel in enumerate(row):
            if pixel == "B":
                car_canvas.set_pixel(x,y,(car_color))
                #print(car_canvas.get_pixel(x, y))
            elif pixel == "W":
                car_canvas.set_pixel(x,y,"gray")
            elif pixel == "T":
                car_canvas.set_pixel(x,y,"dimgray")
            else:
                pass
    return car_canvas



start_server(State([],[],None,""))
```

Chat: I found the problem! It's a small issue in your `color_car()` function, and the error message is pointing us in the right direction.

1. The main problem

Look at this line inside `color_car()`:

```
None
for y, row in enumerate(state.latest_car.body):
```

Your `color_car()` function does not have access to `state`.

Look at its parameters:

```
None
def color_car(car:Car, car_canvas:Picture, car_color:str):
```

You already pass the car into the function through the parameter `car`.

The fix is to use the parameter you already have:

```
None
for y, row in enumerate(car.body):
```

That's it! You don't need to pass `state` into this function.

2. Another small issue I noticed

In your `State` dataclass, you have:

```
None
garage: list[list[str]]
```

But in `make_sports()`, you're doing:

```
None
state.garage.append(state.latest_car)
```

Since `latest_car` is a `Car` object, your garage should actually be typed as:

```
None
garage: list[Car]
```

This is a type annotation issue rather than the cause of your current error.

3. One thing to consider next

I noticed that you have a `COLORS` dictionary:

```
None
COLORS = {    "Blue": "b",    "Red": "r",    "Green": "g",
...}
```

But you're currently passing `car_color` directly into `set_pixel()`.

That's fine if you want to use color names directly, but your dictionary isn't currently

being used.

I'd get your current version working first, then we can look at incorporating that dictionary.

Try changing just the `state.latest_car.body` line to `car.body` and run it again. Your current error should disappear.

Me: it worked but the car is very small

Chat: That's because each character in your car array currently represents just one pixel.

For example, your sports car is approximately 24 pixels wide and 5 pixels tall. Even though your picture is 40 × 40, the car itself only occupies a small portion of it.

The easiest solution is to scale up each pixel.

1. Introduce a scaling variable

For example:

```
None
scale = 5
```

Instead of drawing one pixel for every character, draw a 5 × 5 block of pixels.

2. Adjust your coordinates

Currently, you have:

```
None
car_canvas.set_pixel(x, y, car_color)
```

You want to multiply the coordinates by your scale:

```
None
car_canvas.set_pixel(x * scale, y * scale, car_color)
```

However, this only moves the starting position of each pixel. It doesn't actually make the pixel bigger yet!

3. Use two additional loops

Think about this:

- Your existing loops locate each character in the car array.
- Two additional loops can fill in the pixels surrounding that character.

Conceptually:

```
None
for y, row in enumerate(car.body):    for x, pixel in enumerate(row):        for dy in range(scale):            for dx in range(scale):                # Draw pixel at scaled coordinates
```

Your coordinates would become:

- X: `x * scale + dx`
- Y: `y * scale + dy`

This creates a block of pixels for every character.

4. Increase your picture dimensions

Since you're multiplying the car's dimensions, I'd also change:

```
None
picture = Picture.new(200, 200)
```

A scale of 5 should work nicely with your current car designs.

"Nissan GTR - R34 Skyline - Pixel Car" Poster for Sale by zero260 | Redbubble
redbubble.com

My recommendation: Keep your original car arrays exactly as they are. Scaling the pixels during drawing means you won't have to redesign any of your cars to make them larger!

Me: like this?:

def color_car(car:Car,car_canvas:Picture,car_color:str):

```
    for y, row in enumerate(car.body):
        for x, pixel in enumerate(row):
            for dy in range(scale):
                for dx in range(scale):
                    if pixel == "B":
                        car_canvas.set_pixel(x,y,(car_color))
                        #print(car_canvas.get_pixel(x, y))
                    elif pixel == "W":
                        car_canvas.set_pixel(x,y,"gray")
                    elif pixel == "T":
                        car_canvas.set_pixel(x,y,"dimgray")
                    else:
                        pass


    return car_canvas
```

Chat: You're very close! Your nested loops are correct, but there's one important detail missing.

You're still using:

```
None
car_canvas.set_pixel(x, y, car_color)
```

That only colors the original single coordinate, regardless of how many times your `dx` and `dy` loops run.

You need to incorporate `dx` and `dy` into the coordinates.

Think about this:

If `x = 2`, `y = 3`, and `scale = 5`:

- What should the starting X coordinate be?
- What should the starting Y coordinate be?
- How would `dx` and `dy` change those coordinates as your inner loops run?

The goal is to color a 5 × 5 block instead of just one pixel.

One other thing: make sure your `if`, `elif`, and `else` statements are indented inside

the `dx` loop, so each small pixel gets the appropriate color.

Try making those adjustments yourself, and show me your next version!

Me: scale = 5

```
def color_car(car:Car,car_canvas:Picture,car_color:str):
    for y, row in enumerate(car.body):
        for x, pixel in enumerate(row):
            for dy in range(scale):
                for dx in range(scale):
                    if pixel == "B":
                        car_canvas.set_pixel(x*dx,y*dy,car_color)
                    elif pixel == "W":
                        car_canvas.set_pixel(x*dx,y*dy,"gray")
                    elif pixel == "T":
                        car_canvas.set_pixel(x*dx,y*dy,"dimgray")
                    else:
                        pass
    return car_canvas
```

Chat: Almost! Your loop structure is now correct. There's just a small mathematical issue with your coordinates.

Currently, you're doing:

```
None
car_canvas.set_pixel(x*dx, y*dy, car_color)
```

The problem is that you're multiplying the original coordinates by the loop counters, rather than using the loop counters to expand the pixel.

The adjustment

You want to use:

```
None
x * scale + dxy * scale + dy
```

Here's why:

Imagine `x = 2`, `y = 3`, and `scale = 5`.

- The original pixel starts at `(10, 15)`.
- `dx` moves across the five pixels.
- `dy` moves down the five pixels.

This gives you a 5 × 5 block.

Your corrected drawing calls

None

```
if pixel == "B":    car_canvas.set_pixel(x * scale + dx, y * scale + dy, car_color)elif pixel == "W":    car_canvas.set_pixel(x * scale + dx, y * scale + dy, "gray")elif pixel == "T":    car_canvas.set_pixel(x * scale + dx, y * scale + dy, "dimgray")
```

Everything else about your nested loops can stay the same!

One last thing: Your canvas needs to be large enough for the scaled car. Your current 200 × 200 canvas should be sufficient for your existing designs at scale 5.

You're getting the hang of nested loops here!

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.